LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL [6] – [Inheritance]



NAMA: Fauzi Romadhoni

NIM: 103132400025

KELAS: SISD-05-A

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Pada dasarnya Kita sebagai manusia sudah terbiasa untuk melihat objek yang berada di sekitar Kita tersusun secara hierarki berdasarkan kelas-nya masing-masing. Dari sini kemudian timbul suatu konsep tentang pewarisan yang merupakan suatu proses dimana suatu *class* diturunkan dari kelas lainnya sehingga ia mendapatkkan ciri atau sifat dari kelas tersebut.

B. Guided

1. [Keyword Super]

```
[public class Employee {
   private String name;
   String departemen;

   public Employee(String s) {
      name = s;
   }

   public void tampilNama() {
      System.out.println("nama: " + name);
   }
}]
```

```
[public class Manager extends Employee {
   private String alamat;

   public Manager(String nama, String s) {
      /* Memanggil konstruktor Employee */
      super(nama);
      alamat = s;
   }

   public void tampilAlamat() {
      /* Menginisialisasi variabel departemen yang ada pada superclass */
      super.departemen = "Personalia";

      /* Memanggil method tampilNama() yang ada pada superclass */
      super.tampilNama();

      /* Menampilkan variabel departemen yang telah diinisialisasi */
      System.out.println("alamat: " + alamat);
      System.out.println("departemen: " + super.departemen);
   }

   public static void main(String[] args) {
      /* Membuat objek */
      Manager adi = new Manager("adi", "sukabirus");
      adi.tampilAlamat();
   }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ MODEL6 ---
nama: adi
alamat: sukabirus
departemen: Personalia
------------------------------------------------
[BUILD SUCCESS
```

2. [Overloading Method]

```
[public class Lagu {
    String pencipta;
    String judul;

    // Overloading method versi 1 (1 parameter)
    void IsiParam(String param1) {
        judul = param1;
        pencipta = "Tidak dikenal";
    }

    // Overloading method versi 2 (2 parameter)
    void IsiParam(String param1, String param2) {
        judul = param1;
        pencipta = param2;
    }

    void CetakKeLayar() {
        System.out.println("Judul : " + judul + ", pencipta: " + pencipta);
    }
}]
```

```
[public class DemoOver2 {
    public static void main(String[] args) {
        Lagu d = new Lagu();
        Lagu e = new Lagu();

        d.IsiParam("Lagu 1");
        e.IsiParam("kepastian yang kutunggu", "GiGi");

        d.CetakKeLayar();
        e.CetakKeLayar();
    }
}]
```

Output :

```
] --- exec:3.6.3:exec (default-cli) @ MODEL6 ---
  Judul : Lagu 1, pencipta: Tidak dikenal
- Judul : kepastian yang kutunggu, pencipta: GiGi
  ------------------------------------------------
[ BUILD SUCCESS                                   ]
```

3. [Overriding Method]

```
[class A {
    public void tampilkanKeLayar() {
        System.out.println("Method milik class A dipanggil...");
    }
}

class B extends A {
    @Override
    public void tampilkanKeLayar() {
        super.tampilkanKeLayar(); // Memanggil method milik superclass
        System.out.println("Method milik class B dipanggil...");
    }
}]
```

```
[public class DemoInheritance {
    public static void main(String[] args) {
        B subob = new B();
        subob.tampilkanKeLayar();
    }
}]
```

Output :

```
  Method milik class A dipanggil...
  Method milik class B dipanggil...
  ---------------------------------------
[ BUILD SUCCESS                         ]
```

C. Unguided

1. Soal Unguided 1

   [isi dengan soal unguided 1]

   Jawaban :

   ModelML.java

```
[public class ModelML {
    private String namaModel;
    private int jumlahDataLatih;

    public ModelML(String namaModel, int jumlahDataLatih) {
        this.namaModel = namaModel;
        this.jumlahDataLatih = jumlahDataLatih;
    }

    public String getNamaModel() {
        return namaModel;
    }

    public int getJumlahDataLatih() {
        return jumlahDataLatih;
    }

    public void tampilkanInformasi() {
        System.out.println("Model: " + namaModel + " | Data latih: " +
jumlahDataLatih);
    }

    // Method hitungBiaya (Overloading Versi 1: Tanpa Parameter)
    public int hitungBiaya() {
        return jumlahDataLatih * 2;
    }

    // Method hitungBiaya (Overloading Versi 2: Parameter Epoch)
    public int hitungBiaya(int epoch) {
        return jumlahDataLatih * epoch;
    }
}]
```

   ModelRegresi.java

```
[public class ModelRegresi extends ModelML {
    private int jumlahFitur;

    public ModelRegresi(String namaModel, int jumlahDataLatih, int jumlahFitur)
{
        super(namaModel, jumlahDataLatih);
        this.jumlahFitur = jumlahFitur;
    }
```

```
    @Override
    public void tampilkanInformasi() {
        super.tampilkanInformasi();
        System.out.println("Jumlah fitur: " + jumlahFitur);
    }

    @Override
    public int hitungBiaya() {
        return super.hitungBiaya() + (jumlahFitur * 10);
    }

    // Method hitungBiaya (Overloading Versi 3: Parameter Epoch & Batch Size)
    public int hitungBiaya(int epoch, int batchSize) {
        return getJumlahDataLatih() * epoch / batchSize;
    }
}]
```

ModelKlasifikasi.java

```
    [public class ModelKlasifikasi extends ModelML {
        private int jumlahKelas;

        public ModelKlasifikasi(String namaModel, int jumlahDataLatih, int
    jumlahKelas) {
            super(namaModel, jumlahDataLatih);
            this.jumlahKelas = jumlahKelas;
        }

        public int getJumlahKelas() {
            return jumlahKelas;
        }

        @Override
        public void tampilkanInformasi() {
            super.tampilkanInformasi();
            System.out.println("Jumlah kelas: " + jumlahKelas);
        }

        @Override
        public int hitungBiaya() {
            return super.hitungBiaya() * jumlahKelas;
        }
}
```

ModelDeepLearning.java

```
    [public class ModelDeepLearning extends ModelKlasifikasi {
        private int jumlahLayer;
```

```
    public ModelDeepLearning(String namaModel, int jumlahDataLatih, int
jumlahKelas, int jumlahLayer) {
        super(namaModel, jumlahDataLatih, jumlahKelas);
        this.jumlahLayer = jumlahLayer;
    }

    @Override
    public void tampilkanInformasi() {
        super.tampilkanInformasi();
        System.out.println("Jumlah layer: " + jumlahLayer);
    }

    @Override
    public int hitungBiaya() {
        return super.hitungBiaya() + (jumlahLayer * 200);
    }
}]
```

MainUnguided.java

```
[public class MainUnguided {
    public static void main(String[] args) {
        // Inisialisasi Objek
        ModelRegresi regresi = new ModelRegresi("Regresi Linier", 500, 4);
        ModelKlasifikasi klasifikasi = new ModelKlasifikasi("Decision Tree", 800,
3);
        ModelDeepLearning deepLearning = new ModelDeepLearning("Neural
Network", 1000, 5, 3);

        // Disimpan dalam Array bertipe Class Induk (Polimorfisme)
        ModelML[] daftarModel = {regresi, klasifikasi, deepLearning};

        int totalBiaya = 0;

        // Iterasi Array
        for (ModelML model : daftarModel) {
            model.tampilkanInformasi();
            int biaya = model.hitungBiaya();
            System.out.println("Biaya komputasi: " + biaya);
            System.out.println("---");
            totalBiaya += biaya;
        }

        // Tampilkan Total dan Perhitungan Tambahan
        System.out.println("Total biaya: " + totalBiaya);
        System.out.println("Biaya 10 epoch (Neural Network): " +
daftarModel[2].hitungBiaya(10));
        System.out.println("Biaya 10 epoch (Regresi Linier): " +
regresi.hitungBiaya(10));
        System.out.println("Biaya 10 epoch batch 50 (Regresi Linier): " +
regresi.hitungBiaya(10, 50));
```

```
        }
}]
```

```
--- exec:3.6.3:exec (default-cli) @ MODEL6 ---
Model: Regresi Linier | Data latih: 500
Jumlah fitur: 4
Biaya komputasi: 1040
---
Model: Decision Tree | Data latih: 800
Jumlah kelas: 3
Biaya komputasi: 4800
---
Model: Neural Network | Data latih: 1000
Jumlah kelas: 5
Jumlah layer: 3
Biaya komputasi: 10600
---
Total biaya: 16440
Biaya 10 epoch (Neural Network): 10000
Biaya 10 epoch (Regresi Linier): 5000
Biaya 10 epoch batch 50 (Regresi Linier): 100
------------------------------------------------------------------------
BUILD SUCCESS
```

Penjelasan :

- [ModelML (Superclass): Menyimpan atribut utama (namaModel, jumlahDataLatih) dan menyediakan 2 fungsi hitungBiaya() yang di-overload (tanpa parameter & dengan parameter epoch).
- ModelRegresi : Menambahkan atribut jumlahFitur. Meng-override hitungBiaya dasar dengan rumus ditambah fitur, serta menambah overloading baru dengan parameter epoch dan batchSize.
- ModelKlasifikasi : Menambahkan atribut jumlahKelas dan meng-override hitungBiaya dasar dengan perkalian jumlah kelas.
- ModelDeepLearning : Turunan dari ModelKlasifikasi yang menambahkan jumlahLayer. Meng-override hitungBiaya dengan menambah biaya layer di atas biaya klasifikasi.
- MainUnguided: Menggunakan Polimorfisme untuk menyimpan semua model ke dalam satu array induk ModelML[], lalu menghitung total biaya secara otomatis melalui perulangan.

]

D. Kesimpulan

1. [inheritance (Pewarisan): Memungkinkan kelas turunan (subclass) mewarisi atribut dan method dari kelas induk menggunakan extends untuk mencegah duplikasi kode.
2. Keyword super: Digunakan untuk mengakses konstruktor, method, atau atribut milik kelas induk dari dalam kelas turunan.
3. Overloading vs Overriding:
    - Overloading: Beberapa method bernama sama dalam satu kelas dengan parameter berbeda.
    - Overriding: Kelas turunan mendefinisikan ulang method yang sama persis dengan milik kelas induk.
4. Polimorfisme: Memungkinkan objek dari berbagai kelas turunan ditampung dan diproses melalui satu tipe variabel/array kelas induk.

]